

Relazione Metodi del Calcolo Scientifico

Secondo Progetto

Marelli Samuele 899768 - Fumagalli Andrea 900329

18 giugno 2026

1 Introduzione

Il linguaggio scelto per il progetto risulta essere Python, famoso linguaggio open-source spesso utilizzato per applicazioni scientifiche. Python offre innumerevoli librerie tra cui scegliere per soddisfare il calcolo delle Trasformate di Fourier; abbiamo deciso di usufruire di diverse tra le più famose:

- **NumPy** una delle librerie più utilizzate per la gestione di array e altri costrutti, utilizzata per gestire i segnali tradotti in array nel nostro caso.
- **SciPy**, in particolare il modulo **fft** che contiene funzioni per calcolare le Trasformate di Fourier, da questo modulo importiamo le funzioni *Fast* per calcolare le trasformate ed eseguire la DCT a 1 e 2 dimensioni. In particolare utilizziamo le funzioni **dct** e **dctn**, entrambe con parametro **norm = 'ortho'** per rispettare i valori scalati forniti nel testo.
- **Pillow (PIL)** per caricare e convertire i file **.bmp** in matrici della libreria NumPy.
- **Tkinter** per gestire l'interfaccia utente che permette di scegliere l'immagine da comprimere e i parametri di compressione.

La relazione si struttura in 2 capitoli distinti:

- **DCT2 'homemade'** sezione in cui si mostra la nostra implementazione della DCT1 e DCT2, confrontata successivamente con la versione *Fast* della libreria, discutendo i tempi di esecuzione.
- **Compressione Immagini Stile JPEG**: si discuterà della struttura del nostro codice per realizzare la compressione di una immagine in scala di grigi applicando la DCT2, comparando l'immagine compressa a quella originale e discutendo il risultato ottenuto.

2 Implementazione DCT2 *homemade*

2.1 Libreria di supporto utilizzate

Per la realizzazione della DCT1 e DCT2 non sono state usate librerie per il calcolo delle trasformate di Fourier; sono però state sfruttate alcune funzionalità legate al calcolo matematico e, in particolare, alla gestione degli array grazie rispettivamente alle librerie **Math** e **NumPy**. Di seguito riportiamo la configurazione di queste:

```
1 import math
2 import numpy as np
```

2.2 Codice DCT1

Per l'implementazione della Trasformata Discreta del Coseno (DCT) per matrici quadrate (in 2d) abbiamo deciso di iniziare scrivendo l'algoritmo per calcolare la Trasformata per un singolo vettore monodimensionale (1d), successivamente applicheremo questa funzione alle righe e alle colonne della matrice, come mostrato a lezione. Il codice per la DCT1 è il seguente:

```
1 def dct_1d(segnale):
2     N = np.shape(segnale)[0]
3     # Vettore per memorizzare i coefficienti DCT
4     coeff = np.array([0.0] * N)
5     # Ciclo per le frequenze
6     for k in range(N):
7         somma = 0.0
8         # Ciclo dei singoli campioni del segnale
9         for j in range(N):
10             angolo = k * math.pi * (2 * j + 1) / (2 * N)
11             somma += segnale[j] * math.cos(angolo)
12         # Normalizzazione del coefficiente
13         if k == 0:
14             alpha = math.sqrt(1.0 / N)
15         else:
16             alpha = math.sqrt(2.0 / N)
17         coeff[k] = alpha * somma
18     return coeff
```

La funzione utilizza un doppio ciclo for, il primo serve per selezionare la singola frequenza k su cui operare in quella iterazione, il secondo ciclo for invece calcola il coefficiente della frequenza in analisi. Prima di calcolare il coefficiente si stabilisce il fattore di normalizzazione da utilizzare per quella iterazione. Una volta calcolati i coefficienti vengono ritornati sotto forma vettoriale.

2.3 Codice DCT2

Come anticipato, il calcolo della DCT2 homemade si basa sull'applicazione della DCT1 prima alle righe della matrice originale, e successivamente alle colonne della matrice risultante.

```

1 def dct_2d(matrice):
2     num_righe = np.shape(matrice)[0]
3     num_colonne = np.shape(matrice)[1]
4     matrix_temp = np.zeros((num_righe, num_colonne))
5     # Applica dct1 su ogni riga
6     for i in range(num_righe):
7         matrix_temp[i, :] = dct_1d(np.squeeze(matrice[i, :]))
8     matrix_final = np.zeros((num_righe, num_colonne))
9     # Applica dct1 su ogni colonna
10    for j in range(num_colonne):
11        colonna = np.squeeze(matrix_temp[:, j])
12        matrix_final[:, j] = dct_1d(colonna)
13    return matrix_final

```

Otteniamo il numero di righe e colonne della matrice originale, creiamo una nuova matrice intermedia, inizializzata a zero, basata sulla dimensione di quella originale all'interno della quale verranno immagazzinati i coefficienti della DCT1 applicata sulle righe. Calcoliamo questi coefficienti applicando la DCT1 sulle righe; definiamo la matrice finale, che conterrà i coefficienti corretti, inizializzandola con zero. La matrice finale sarà riempita con i coefficienti calcolati applicando la DCT1 alle righe della matrice intermedia calcolata al passo precedente. Infine ritorniamo la matrice finale dei coefficienti.

2.4 Validazione algoritmo

Per assicurarci che il calcolo risulti corretto abbiamo confrontato i coefficienti calcolati da entrambe le DCT: sia quella *homemade* che quelle implementate nelle librerie; mettendoli a paragone con quelli calcolati su una matrice nota. Per la correttezza dei coefficienti dell'algoritmo DCT1, utilizziamo il vettore:

$$\begin{bmatrix} 231 & 32 & 233 & 161 & 24 & 71 & 140 & 245 \end{bmatrix}$$

I coefficienti dati dall'applicazione della DCT1 a questo vettore devo essere:

$$\begin{bmatrix} 4.01e+02 & 6.60e+00 & 1.09e+02 & -1.12e+02 & 6.54e+01 & 1.21e+02 & 1.16e+02 & 2.88e+01 \end{bmatrix}$$

Eseguendo gli algoritmi notiamo che:

- la DCT1 homemade ritorna i corretti coefficienti grazie all'impostazione di normalizzazione.
- la libreria invece necessita della specifica `norm = 'ortho'`, per comunicare alla funzione di non utilizzare la normalizzazione di default (backward), ma applicare quella ortonormale.

Per la correttezza della DCT2 invece, utilizziamo la matrice:

$$\begin{bmatrix} 231 & 32 & 233 & 161 & 24 & 71 & 140 & 245 \\ 247 & 40 & 248 & 245 & 124 & 204 & 36 & 107 \\ 234 & 202 & 245 & 167 & 9 & 217 & 239 & 173 \\ 193 & 190 & 100 & 167 & 43 & 180 & 8 & 70 \\ 11 & 24 & 210 & 177 & 81 & 243 & 8 & 112 \\ 97 & 195 & 203 & 47 & 125 & 114 & 165 & 181 \\ 193 & 70 & 174 & 167 & 41 & 30 & 127 & 245 \\ 87 & 149 & 57 & 192 & 65 & 129 & 178 & 228 \end{bmatrix}$$

Questa deve ritornare i coefficienti:

$$\begin{bmatrix} 1.11e+03 & 4.40e+01 & 7.59e+01 & -1.38e+02 & 3.50e+00 & 1.22e+02 & 1.95e+02 & -1.01e+02 \\ 7.71e+01 & 1.14e+02 & -2.18e+01 & 4.13e+01 & 8.77e+00 & 9.90e+01 & 1.38e+02 & 1.09e+01 \\ 4.48e+01 & -6.27e+01 & 1.11e+02 & -7.63e+01 & 1.24e+02 & 9.55e+01 & -3.98e+01 & 5.85e+01 \\ -6.99e+01 & -4.02e+01 & -2.34e+01 & -7.67e+01 & 2.66e+01 & -3.68e+01 & 6.61e+01 & 1.25e+02 \\ -1.09e+02 & -4.33e+01 & -5.55e+01 & 8.17e+00 & 3.02e+01 & -2.86e+01 & 2.44e+00 & -9.41e+01 \\ -5.38e+00 & 5.66e+01 & 1.73e+02 & -3.54e+01 & 3.23e+01 & 3.34e+01 & -5.81e+01 & 1.90e+01 \\ 7.88e+01 & -6.45e+01 & 1.18e+02 & -1.50e+01 & -1.37e+02 & -3.06e+01 & -1.05e+02 & 3.98e+01 \\ 1.97e+01 & -7.81e+01 & 9.72e-01 & -7.23e+01 & -2.15e+01 & 8.13e+01 & 6.37e+01 & 5.90e+00 \end{bmatrix}$$

Analogamente al caso precedente, la DCT1 homemade ritorna i coefficienti corretti per costruzione, mentre quella derivata dalla libreria necessita del parametro `norm = 'ortho'` nella chiamata di funzione.

2.5 Confronto tempi di esecuzione

Come passaggio finale, abbiamo confrontato i tempi di esecuzione della nostra implementazione con la versione *Fast* della libreria **SciPy**. Per valutare le tempistiche abbiamo generato matrici quadrate di dimensione $N \times N$ con N crescente, eseguendo entrambi gli algoritmi e misurando il tempo impiegato per calcolare le trasformate. Di seguito riportiamo una tabella con i vari tempi di esecuzione per i diversi valori di N :

Dimensione	DCT2 Homemade	DCT2 SciPy
$N = 4$	0.000113 s	0.000036 s
$N = 8$	0.000448 s	0.000030 s
$N = 16$	0.003077 s	0.000035 s
$N = 32$	0.019871 s	0.000070 s
$N = 64$	0.140704 s	0.000093 s
$N = 128$	1.199802 s	0.000170 s
$N = 256$	10.125923 s	0.000477 s

Per visualizzare meglio l'andamento asintotico abbiamo inserito i dati su un grafico a scala semilogaritmica sulle ordinate.

Come mostrato dall'andamento del grafico, i tempi riflettono la complessità teorica. La nostra implementazione della DCT2 possiede una crescita cubica, il tempo è quindi proporzionale a N^3 ; l'algoritmo di SciPy invece sfrutta l'ottimizzazione *fft* (Fast Fourier Transform) con tempi di esecuzione proporzionali a $N^2 \log(N)$, risultando quindi ordini di

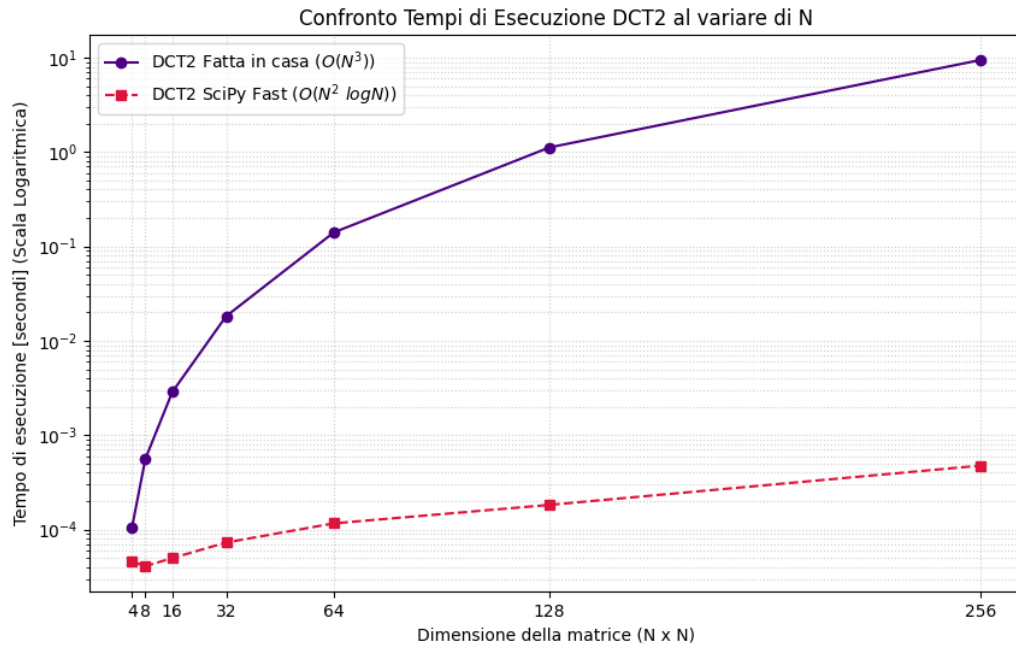


Figura 1: Confronto tempi di esecuzione al variare di N

grandezza più efficiente al crescere di N . Una anomalia segnalata è la discrepanza con valori di N molto piccoli, infatti quando le matrici (e quindi le operazioni) risultano di scarsa dimensione, l'implementazione di **SciPy** tende ad essere irregolare, atteggiamento dovuto all'overhead aggiunto dalla chiamata alla libreria (trascurabile sulle grandi dimensioni).

3 Compressione immagini stile JPEG

3.1 Struttura del codice

Il secondo obbiettivo del progetto è applicare la DCT2 con lo scopo di comprimere immagini in scale di grigi, ricostruendo successivamente quest'ultima notando la perdita di informazioni. Per motivi di efficienza e velocità la compressione usando DCT2 non avviene con il metodo *homemade* precedentemente sviluppato, ma si utilizza la versione *Fast* presente nella libreria **SciPy**. Il codice deve permettere un comportamento simile all'algoritmo di compressione JPEG; il progetto è così strutturato:

- **interfacciaImmagini.py**: si tratta della classe che gestisce l'interfaccia utente, ha il compito di interagire con quest'ultimo e mostrare i risultati dell'algoritmo applicato all'immagine scelta.
- **compressioneImmagini.py**: è la classe che si occupa di eseguire l'effettiva compressione di una matrice (immagine in scala di grigi convertita); la classe accetta parametri quali ampiezza (**f**) e soglia di taglio (**d**). Permette anche di ottenere la matrice ricostruita ottenuta dalla divisione, compressione e filtraggio della matrice originale.
- **main.py**: si tratta del file principale, da qui viene eseguito il codice per la comparazione tra *DCT homemade* sviluppata da noi e *DCT Fast* della libreria. Successivamente a questa comparazione viene inizializzata l'interfaccia che si occuperà di gestire la sezione di compressione immagini in stile jpeg.

3.2 Interfaccia grafica Tkinter

Per gestire l'interfaccia utente abbiamo usufruito di una famosa libreria standard di Python: **Tkinter**. Questa ci ha permesso di creare una semplice finestra che permettesse all'utente di inserire i valori di ampiezza e soglia di taglio e l'immagine da comprimere. Nell'interfaccia è presente un pulsante che avvia la verifica dei valori inseriti: se l'immagine risulta idonea e se ampiezza e taglio rispettano le regole imposte; le variabili risultano corrette ed il programma procede alla compressione dell'immagine. La compressione avviene asincronamente grazie alla creazione di un nuovo Thread che si occuperà di chiamare la classe **compressioneImmagini**. Abbiamo ritenuto utile impostare l'esecuzione in questa maniera per poter osservare il risultato finale su diversi esempi contemporaneamente, beneficio aggiunto deriva dal poter interagire con l'interfaccia una volta avviata la compressione.

3.2.1 Caricamento immagine

La funzione `load_img` si occupa di caricare l'immagine fornita come percorso assoluto dalla GUI; analizzando il codice e notiamo che forziamo l'immagine ad assumere una scala di grigi con il comando `Image.open().convert('L')`, vitale siccome ci occupiamo unicamente di matrici a due dimensioni, dunque non colorate. Per convertire l'immagine in una matrice sfruttiamo un combinazione di **Pillow** per caricarla e forzarla in scala di grigi, e **NumPy** per creare un oggetto array facilmente accessibile.

```

1 def load_img(self, path):
2     try:
3         img = Image.open(path).convert('L')
4         return np.array(img)
5     except Exception as e:
6         print(f"Errore nel caricamento dell'immagine: {e}")
7         return None

```

3.3 Compressione

3.3.1 Taglio matrice

La funzione `matrix_cut()` permette di tagliare la matrice originale in modo che la risultante sia un multiplo di f per le dimensioni di riga e colonna originali, tutti i dati che risultano oltre andranno scartati. Questo velocizza la successiva fase di divisione in blocchi di dimensione $f \times f$, in quanto le dimensioni della matrice saranno multipli di f stesso.

```

1 def matrix_cut(self, matrix):
2     H, W = matrix.shape
3     cut_H = (H // self.f) * self.f
4     cut_W = (W // self.f) * self.f
5     cropped_matrix = matrix[:cut_H, :cut_W]
6     print(f"Dimensioni originali: {H}x{W}")
7     print(f"Dimensioni ritagliate: {cut_H}x{cut_W}")
8     return cropped_matrix

```

3.3.2 Suddivisione in blocchi, DCT e filtraggio frequenze

La divisione in blocchi è effettuata dalla funzione `split_in_blocks()` che divide una matrice passata come parametro in blocchi di dimensione $f \times f$, assumendo che la matrice abbia numero di colonne e righe multipli di f stesso. Ritorna un array di **NumPy** che contiene i vari blocchi, si tratta di un array tridimensionale, in quanto ogni elemento dell'array è in sé una matrice a due dimensioni (il singolo blocco).

```

1 def split_in_blocks(self, matrix):
2     H, W = matrix.shape
3     blocks = []
4     for i in range(0, H, self.f):
5         for j in range(0, W, self.f):
6             block = matrix[i:i+self.f, j:j+self.f]
7             blocks.append(block)
8     return np.array(blocks)

```

Il lavoro di compressione è delegato alle due funzioni `process_block()` e `process_blocks()`, la prima si occupa di processare il singolo blocco, la seconda chiamerà la prima funzione

su ogni blocco e accumulerà i risultati. `process_block()` applica inizialmente la DCT2 a tutto il blocco, ottenuti i coefficienti li analizza scartando tutti quelli in cui è vero $c_{lk} \geq d$ ed impostando il loro valore a 0. Una volta rimossi i coefficienti applica la IDCT2 alla matrice risultante; infine arrotonda ogni coefficiente all'intero più vicino, impostando quelli negativi = 0 e quelli > 255 a 255. Ritorna il vettore calcolato. `process_blocks()` applica la funzione precedente ad ogni blocco, accumulando in una matrice tridimensionale i risultati; ritorna questa matrice una volta elaborati tutti i blocchi.

```

1  def process_block(self, block):
2      c = dctn(block, type=2, norm='ortho')
3      for row in range(self.f):
4          for col in range(self.f):
5              if (row + col) >= self.d:
6                  c[row, col] = 0
7      ff = idctn(c, type=2, norm='ortho')
8      ff_int = np.clip(np.round(ff).astype(int), 0, 255)
9      return ff_int
10
11 def process_blocks(self, blocks):
12     processed_blocks = []
13     for block in blocks:
14         processed_block = self.process_block(block)
15         processed_blocks.append(processed_block)
16     return np.array(processed_blocks)

```

3.3.3 Ricostruzione

La ricostruzione avviene inserendo in posizione corretta i valori presenti in ogni blocco processato; i blocchi vengono posizionati in modo da ricostruire una riga alla volta.

```

1  def reconstruct_matrix(self, blocks, original_shape):
2      H, W = original_shape
3      res = np.zeros((H, W), dtype=int)
4      i = 0
5      for row in range(0, H, self.f):
6          for col in range(0, W, self.f):
7              res[row:row+self.f, col:col+self.f] = blocks[i]
8              i += 1
9      return res

```

3.3.4 Compressione

La funzione principale non esegue operazioni essa stessa, ma si limita a chiamare le altre funzioni menzionate nel corretto ordine per ottenere il risultato corretto. Ritorna la matrice bidimensionale che corrisponde all'immagine originale compressa, la GUI si occuperà successivamente di mostrarla.

```

1 def compress_matrix(self, matrix):
2     cropped_matrix = self.matrix_cut(matrix)
3     blocks = self.split_in_blocks(cropped_matrix)
4     processed_blocks = self.process_blocks(blocks)
5     reconstructed_matrix = self.reconstruct_matrix(processed_blocks, cropped_matr
6     return reconstructed_matrix

```

3.4 Analisi risultati

Mostriamo alcuni esempi per diverse immagini, analizzando cosa succede sulle singole e nell'insieme.

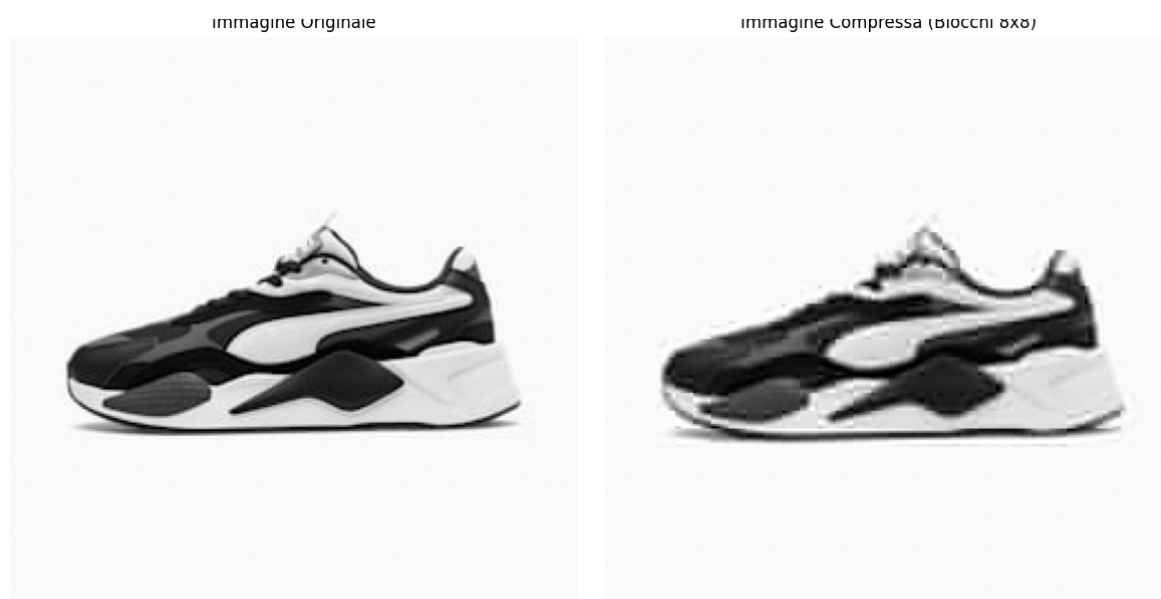


Figura 2: Immagine 260×260 con $d = 5$ e $f = 8$

Mantenendo costanti l'ampiezza del blocco f e la soglia di taglio d notiamo che più l'immagine si ingrandisce e meno la perdita di informazioni è impattante. Nell'immagine della scarpa la compressione è notevole, la perdita di alcuni coefficienti impatta molto data la ristretta dimensione. Nella seconda invece, per quanto ancora visibile e percettibile, la perdita si riduce maggiormente, ricordando che l'immagine possiede circa 6 volte tanti pixel ($260 \times 260 = 67600$ vs $450 \times 800 = 360000$). Per il ponte (immagine 3) la stessa compressione risulta praticamente invisibile all'occhio nudo, data l'enorme quantità di informazioni che l'immagine possiede. Si arriva quindi alla conclusione che maggiore è il numero di pixel presenti nell'immagine, minore è la perdita di informazioni percepita.



Figura 3: Immagine 450×808 con $d = 5$ e $f = 8$

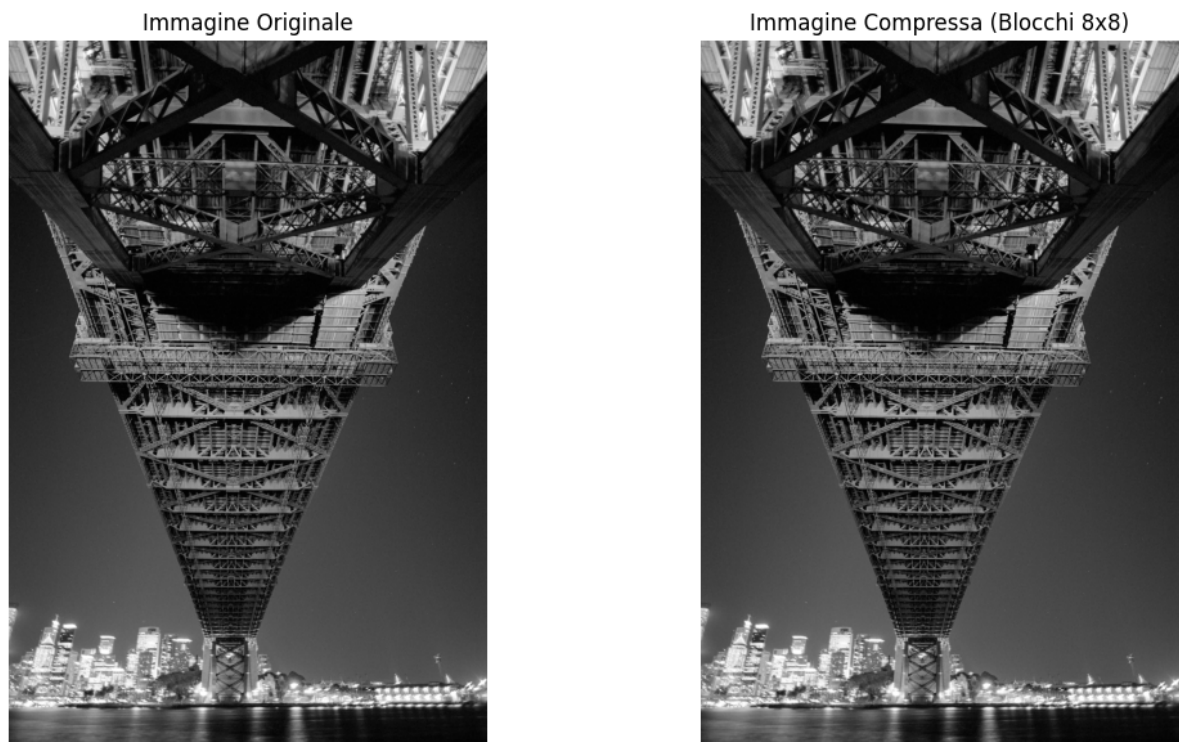


Figura 4: Immagine 4049×2749 con $d = 5$ e $f = 8$

Mostrando altri esempi per l'immagine del ponte notiamo che:

- **figura 5** $d = 1$ e $f = 8$, nonostante si perdano tutte le informazioni del blocco tranne un singolo coefficiente, la compressione risulta sicuramente visibile ma meno marcata di quanto aspettato.
- **figura 6** $d = 1$ e $f = 15$, aumentando la grandezza dei blocchi a 15×15 ma mantenendo la soglia di taglio a 1, la perdita di informazioni comincia a diventare notevole, con dettagli dell'immagine originale difficilmente visibili e distinguibili.

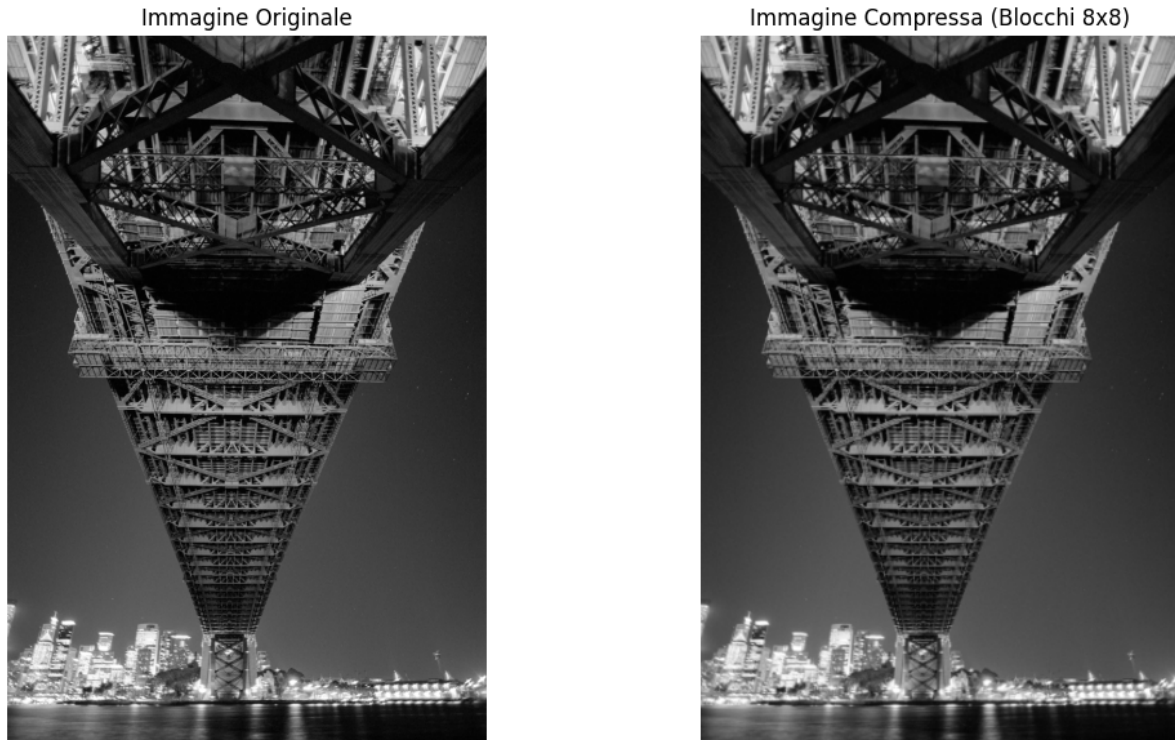


Figura 5: Immagine 4049×2749 con $d = 1$ e $f = 8$

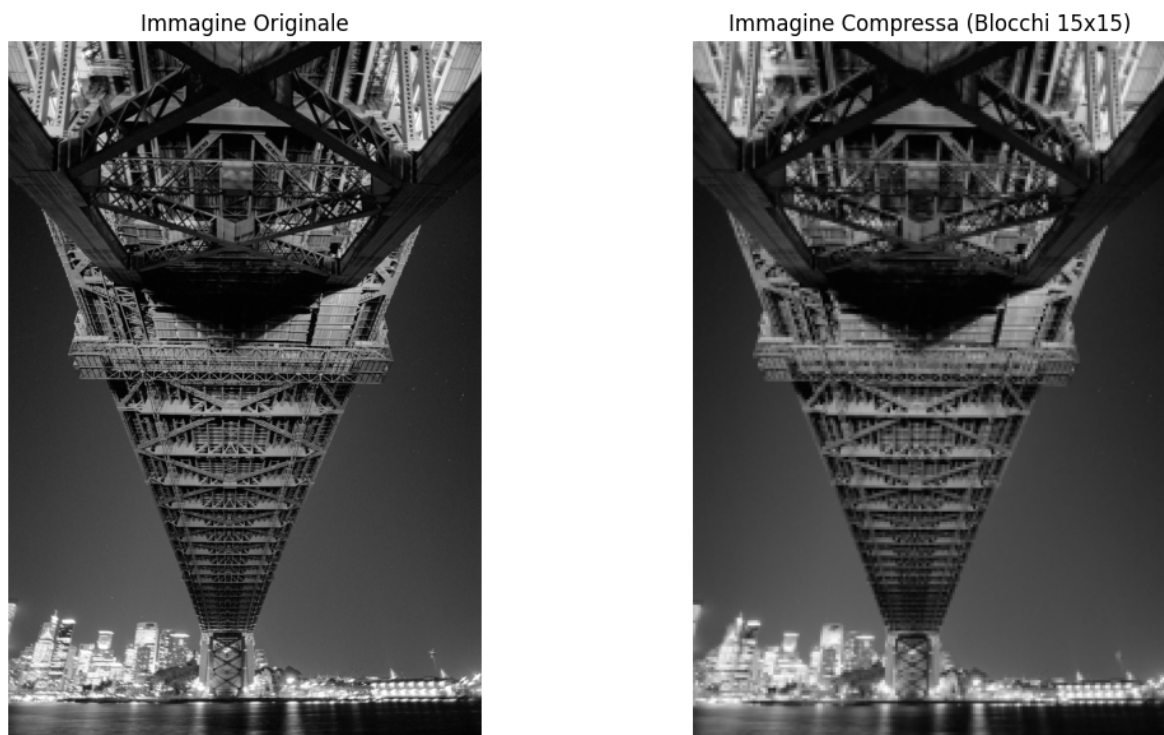


Figura 6: Immagine 4049×2749 con $d = 1$ e $f = 15$